

Project Proposal

AgentDesign: Hermes Profile Architecture

Math & Librarian Profiles — Design Documenta

Mehdi Ghasemi

Project Proposal

September 2026

Executive Summary

This document records the complete design of the two Hermes agent profiles that power my mathematical research workflow — `math` and `librarian` — and specifies an abstraction layer under which either profile can be re-pointed at any future math project without modification. Sanitized, ready-to-use copies of both profiles ship with this document as the `math/` and `librarian/` folders (Section 4); the companion artifacts listed in the closing Sources and Verification note let every claim here be re-derived from a live installation.

The architecture resolves into five layers: a shared *Hermes core*, an isolated *profile system*, a *skill catalog* (capability-gated per profile), an *MCP bridge* (native servers plus a generic CLI-adapter pattern), and an *external service plane*. The math profile is a full-stack researcher with a sandboxed toolchain environment; the librarian profile is a capability-degraded derivative that shares every registration but disables the computational surface and adds ingestion skills of its own.

Two structural findings drive the abstraction design: (i) the librarian profile's MCP servers import scripts from the math profile's directory tree, so it cannot exist without the math profile; and (ii) both profiles carry a stale reference to a development worktree that no longer exists (a merge left the registered path dangling). Section 10 proposes a capability/manifest model — persona templates, path interpolation, a shared adapter, a service registry, and per-project binding files — that removes every identified coupling in five concrete steps.

Contents

1 Objectives and Scope	3
2 Layered Architecture	3
3 Profile System: Structure Comparison	4
4 Folder Structure	5
5 Skill System: Design for General Math Projects	7
6 MCP Bridge	9
7 Tool Stack and Service Plane	10
8 The Research Workflow	11
9 Identified Couplings and Defects	11
10 Toward an Abstraction Layer	13
11 Roadmap for AgentDesign Itself	15
A The Self-Hosted Service Plane	16
A.1 Component inventory	16
A.2 Role in the six-stage workflow	18
A.3 Dependencies outside the stack	18
A.4 Deployment notes	19
B The Model Context Protocol Bridge	19
B.1 Conventions of the bridge	19
B.2 What each server exposes	21
B.3 Notes on the bridge	22

I Objectives and Scope

The `AgentDesign` project has three objectives:

1. **Document the as-is architecture.** Record, with file-level evidence, how the math and librarian profiles are composed: directory layout, configuration sections, skill catalogs, MCP server registrations, external services, scheduled jobs, and state stores. This is Sections 2–8.
2. **Identify the coupling points.** Enumerate every place where a profile depends on the specific current project layout — absolute paths, cross-profile imports, hardcoded endpoints, and stale references — so that the design of record distinguishes what is load-bearing from what is incidental (Section 9).
3. **Specify an abstraction layer.** Define how profiles become reusable personas: independent of any particular math project, instantiable for a new project with zero or one line of configuration, and testable against a capability contract (Section 10).

The scope is the two active research profiles. The remaining four registered profiles (`default`, `coder`, `advisor`, `trader`) are out of scope; their existence is noted only in Section 3. This project does not modify any live profile configuration — it is a documentation and design artifact.

2 Layered Architecture

Figure 1 gives the visual layout of the infrastructure under which both profiles operate. Five layers are distinguished; each lower layer is shared, each upper layer is isolated per profile at `~/.hermes/profiles/<name>/`.

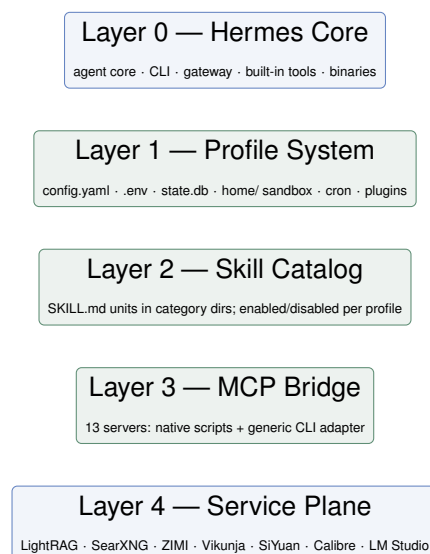


Figure 1: Five-layer profile architecture. Layers 2–3 are the capability surface of a profile; layers 0 and 4 are shared infrastructure. Arrows indicate the direction in which each layer is configured by, or depends on, the layer above it.

Layer 0 — Hermes Core

The core lives at `~/.hermes/hermes-agent/`: agent runtime, CLI (profile switching, tool registry), platform gateway (Telegram/WhatsApp/webhook adapters), built-in tools (terminal, browser, vision, file I/O, web

search, kanban, memory) and the MCP discovery layer (`tools/mcp_tool_*.py`). Shared binaries (`tirth` security scanner, `uv`, Python 3.14, Node 26, `ripgrep`, Chromium) live in `~/.hermes/tools/`; all MCP stdio servers run from a single shared virtualenv at `~/.hermes/mcp-servers/.venv/`.

Layer 1 — Profile System

Each profile is an isolated directory with its own configuration, credentials, state and sandbox. The two profiles under study:

- `math` — full-stack mathematical researcher. It owns a sandboxed home environment carrying the local toolchain (Section 3), an active messaging gateway, and the scheduled literature jobs.
- `librarian` — literature manager (“manage and maintain books and articles”). It keeps an empty sandbox home, registers no scheduled jobs and no messaging platform, and reuses the same kanban assignment as `math` — itself a coupling noted in Section 9.

Both profiles run the same local model through LM Studio, with an `on_session_start` hook that guarantees the model server is up before each session. Long-term memory in both profiles is backed by Honcho (`memory.provider: honcho`).

Layer 2 — Skill Catalog

Skills are Markdown-defined capability units (`SKILL.md` with YAML frontmatter), organized in category directories, optionally carrying scripts and credential files. Each profile holds the same catalog — nine populated categories plus a handful of top-level skills, and the same empty category shells — and differs only in which entries are activated. The per-profile inventory extracted from a live installation is kept in `references/skill_inventory.md`; Section 5 summarizes the design-relevant parts.

Layer 3 — MCP Bridge

Skills reach services either directly (terminal/shell) or through Model Context Protocol servers registered in each profile’s config; every registered tool is exposed to the model under a name built from the server and the tool, joined by double underscores — for example, the `vikunja` task-creation tool. Section 6 documents the two server families and the cross-profile dependency this layer creates.

Layer 4 — Service Plane

A fixed set of self-hosted services on the local network, plus SaaS APIs and a local inference server. Table 4 lists them with their role in the workflow.

3 Profile System: Structure Comparison

Table 1 compares the two profiles as designs rather than as installations: the question is what each profile *is*, not what it happens to contain today.

Table 1: Profile design comparison: the librarian is a capability-degraded derivative of the math profile, not an independent design.

Design element	math	librarian
Role	Full-stack mathematical researcher	Literature manager: “manage and maintain books and articles”
Inference	Local model served by LM Studio	Same model and provider, with an explicit API mode
Skill surface	Full catalog; computation, formal proof and manuscript skills active	Same catalog with the computational surface disabled in configuration; adds its own ingestion skills (<code>calibre-library-ingestion</code> , <code>djvu-to-pdf</code>)
MCP bridge	Native servers plus adapter-wrapped CLI tools	Identical registrations, all resolving into the math profile’s tree
Plugins	<code>vikunja</code> and <code>wiki-browser</code> enabled	Identical
Toolchain sandbox	Lean 4, Rust, SageMath and TeX Live reachable from the profile home	Empty stub
Scheduled jobs	Literature scan and wiki maintenance	None
Messaging platform	Active gateway	None
Cross-session memory	Honcho	Honcho
Kanban assignment	math	math — inherited, not chosen

The librarian is a capability-degraded derivative of math, not an independent design: it inherits every skill directory and MCP registration, then suppresses the computational surface via the `skills.disabled` list. It also carries two skills that exist only in its own tree — `calibre-library-ingestion` and `djvu-to-pdf` (productivity category) — which is where its genuine specialization lives.

The math profile carries a sandboxed home directory that stands in for a machine-local toolchain. It holds a Lean 4 installation managed by `elan`, a Rust toolchain together with its registry and download caches, a dedicated browser profile for NotebookLM grounded synthesis, runtime state directories for SageMath and Maxima, and the font and format caches of TeX Live. Two properties matter for the abstraction design. First, most of the space is disposable package cache rather than toolchain state: the load-bearing parts are the Lean and Rust installations, everything else can be re-provisioned. Second, the sandbox is not self-contained — the SageMath and TeX Live binaries resolve to host-level installations rather than to anything inside the profile home — so a profile’s computational capability depends on the host even when it looks self-contained.

4 Folder Structure

Three directory levels carry the design: the shared Hermes root, the two profile trees, and the per-project binding file. The listings below show the structure with the design-relevant entries; the profile skeleton in Listing 4 marks which entries exist in which profile.

Shared root

The `~/hermes/` root holds everything that is not profile-scoped. Two entries are load-bearing for the coupling analysis: `mcp-servers/` holds the single Python environment that every MCP server of every profile runs in,

and `skills/` is the shared pool that both profiles fall back to for both skills and plugins.

```

1 ~/.hermes/                # shared Hermes root
2  config.yaml              # global defaults; profiles override
3  profiles/                # one directory per profile (math, librarian, advisor, ...)
4  skills/                  # shared skill pool: 17 categories
5  plugins/                 # shared plugin installs: vikunja, wiki-browser, hermes-
    achievements
6  desktop-plugins/         # desktop plugin surfaces: vikunja, wiki-browser
7  plugin-data/             # per-plugin persistent state: hermes-achievements
8  mcp-servers/.venv/       # ONE Python env for all 13 MCP servers
9  memories/ sessions/ state.db # memory files, session store, global state
10 kanban/ kanban.db        # kanban board + dispatcher state
11 cron/ hooks/ agent-hooks/ # scheduling and lifecycle hooks
12 bin/ node/ webui/ tools/  # runtime binaries, desktop build, CLI tools

```

Profile tree: the two profiles side by side

The math profile is a structural superset: it carries the runtime residue of an active profile, while the librarian keeps only what its role needs. Two differences are design-relevant — `scripts/`, the MCP adapter directory, exists only on the math side, and `home/` is populated only there. Those two facts are the structural root of the path coupling in Section 6.

1	profiles/math/		profiles/librarian/
2	config.yaml	profile.yaml	config.yaml profile.yaml
3	.env		.env
4	skills/	24 top-level dirs	skills/ 23 top-level dirs
5	plugins/	vikunja, wiki-browser,	plugins/ empty (borrows shared pool)
6		hermes-android (disabled)	
7	desktop-plugins/	vikunja, wiki-browser	desktop-plugins/ [absent]
8	scripts/	mcp_cli_adapter.py + 8 .py	scripts/ [absent]
9	home/	toolchain sandbox	home/ (empty stub)
10	scratchpad/	failure scratchpad	scratchpad/ [absent]
11	cron/ hooks/ memories/ sandboxes/		cron/ hooks/ memories/ sandboxes/
12	sessions/ workspace/ cache/ logs/		sessions/ workspace/ cache/ logs/
13	backups/ state.db SOUL.md		backups/ state.db SOUL.md
14	telemetry/	[absent]	telemetry/ (sender on)
15	vault/	[absent]	vault/ (credential store)
16	state.db / SOUL.md / backups/		state.db / SOUL.md / backups/

Beyond the entries shown, the librarian keeps `telemetry/` and `vault/`; math's extra entries are dominated by runtime residue (gateway sockets, backup snapshots, retired write-ahead logs) plus the three design-relevant directories listed above.

Skill tree layout

Both profiles carry the same 24-directory layout except for the single extra top-level skill in math; activation, not layout, is what differs.

```

1 profiles/<profile>/skills/ # 24 top-level dirs (math) / 23 (librarian)
2  research/ productivity/ mlops/ # 9 populated categories; each holds
3  software-development/ github/ devops/ # <skill>/SKILL.md with optional
4  web/ autonomous-ai-agents/ social-media/ # scripts/ and references/
5  apple/ creative/ data-science/ email/ # 9 empty category shells (reserved,
6  environment/ media/ note-taking/ # no SKILL.md; used by the curator

```

```

7  mathematical-research/ smart-home/          # as retirement targets)
8  differential-sdp/ ghasemi-latex-style/       # 6 top-level skills in math,
9  hermes-plugin-management/ semantic-scholar/# 5 in librarian
10 workspace-doc-hierarchy/ irene-rewrite-dev/ [math only]
11 .archive/                                     # retired skills kept as curator tombstones
12 .curator_ledger.jsonl .curator_state/ .curator_suppressed/
13 .bundled_manifest .usage.json .hub/ .locks/

```

Distributable profile packs

The `math/` and `librarian/` folders inside this project are sanitized, ready-to-use copies of the two profiles: the same directory structure, the same configuration and skill trees, with every credential, endpoint, hostname and local path replaced by a placeholder. A builder script in `references/` copies the configuration, skill, plugin, script and memory trees from a live installation, scrubs secrets and identity, and writes a per-pack `README.md`. Copying a pack into `~/.hermes/profiles/` and filling in the placeholders reproduces the profile; the packs are described in Section 4.

This document's own folder

```

1 AgentDesign/
2  agent_design.tex          # this document (source of record)
3  agent_design.pdf         # compiled output
4  AGENTS.md                # project guidance for future sessions
5  math/ librarian/         # sanitized, ready-to-use profile packs (Section~\ref{sec:
    packs})
6  stack/                  # the profiles' service plane as one compose project
7  cpistuff/               # CPI LaTeX template: cpi.sty + logos
8  t1ggm.fd ts1ggm.fd      # Garamond T1 declarations (compile prerequisite)
9  data/profile_comparison.json # machine-readable extraction (secret values redacted)
10 references/extract_profile_data.py references/build_profile_packs.py
11 references/sanitize_extracted_data.py
12 references/skill_inventory.md # per-profile skill inventory
13 .gitignore

```

The `stack/` folder turns the service plane of Section 7 into an executable artifact: one compose project that brings up every container the two profiles talk to, with credentials interpolated from an untracked `.env`. Its `README.md` maps each container to the MCP server or skill that consumes it and records what the live deployment gets wrong.

Reproducing the build requires one environment step beyond the template: the CPI style loads the Garamond T1 fonts, which are not part of a stock TeX Live installation. They are installed per-user (no root) under `~/texmf/` — `ggm*.tfm` in `fonts/tfm/ggm/`, `ggm*.vf` in `fonts/vf/ggm/`, `ggm*.pfb` in `fonts/type1/ggm/`, `ggm.map` in `dvips/ggm/`, then `mktexlsr ~/texmf`. Without it the build fails at the first page with a missing-font error rather than a document error.

5 Skill System: Design for General Math Projects

Skills are the unit of reusability. Each `SKILL.md` carries a trigger description in its frontmatter, an imperative body, optional scripts, and — where needed — a credential file (`.env`) co-located with the skill that consumes it. The catalog is shared between the two profiles: the same populated categories, the same empty category shells, and the same top-level skills, with a single exception — `irene-rewrite-dev` is an active top-level skill

in math and a curator tombstone in the librarian. Only activation differs. Table 2 summarizes the catalog by design role; the per-entry inventory extracted from a live installation is in `references/skill_inventory.md`.

Table 2: Skill catalog by category, with activation per profile.

Category	Design role	Activation
research	The research surface proper: the six-stage workflow, symbolic and numeric computation, formal verification, literature retrieval and the wiki pipeline	full in math; computational and proof skills disabled in librarian
productivity	Knowledge management: task ledger, notes, bibliography, manuscript production, document conversion	full in both; the librarian adds its ingestion skills
mlops	Local inference operations and pipeline tooling	reduced in librarian
software-development	Hermes self-extensibility, planning, subagent orchestration, desktop inspection	full in math; partly disabled in librarian
github, devops, agents, web	Repository operations, SDLC review, delegation, blocked-page recovery	mixed; review and delegation off in librarian
social-media	Community reading	inverted gate: disabled for math, enabled for librarian
top-level skills	Cross-cutting capabilities: differential-algebraic SDP, manuscript style, plugin management, citation graph, document hierarchy	full in math; the librarian disables the development-oriented entry

Three observations are design-relevant:

1. **The math research surface is exactly the set of skills a generic math project needs:** decompose → literature → compute → verify → write, implemented by `math-research-workflow`, whose frontmatter names the sibling skills it relies on. This set, not the full catalog, defines the reusable “mathematician” capability contract (Section 10).
2. **The librarian’s disabled list is a negative specification of that same contract:** every computational skill (`sagemath-mcp`, `sympy-mcp`, `lean4`, `lean4-goedel-agent`, `scientific-coding`), the DSDP skills, and the Irene/dev skills are turned off. Capability gating by negation works today but is brittle in a subtler way than it looks: entries resolve by *leaf name* against the profile’s skill tree, which includes the curator tombstone directory `skills/.archive/`. Every disabled name therefore resolves to a real `SKILL.md`; none dangles. But a sizeable minority of them resolve *only* under `.archive/` — they are residue from skills the curator has already retired (`docx`, `opencode`, `test-driven-development`, and for the librarian also `irene-rewrite-dev`, which is an *active* top-level skill in math while pointing at a librarian tombstone). Because resolution is name-based, any of these stale entries would silently disable a same-named skill if it were ever reinstalled. One inversion deserves note: `reddit-reading` is disabled for math but enabled for librarian — the only skill whose gate runs opposite to every other difference between the two profiles.
3. **One reference is stale:** `math-research-workflow` names `simplerag-memory` in its frontmatter and body, several times as shell invocations of a client script that does not exist; no such skill exists in either profile tree or in the bundled pool.

6 MCP Bridge

Thirteen MCP servers are registered per profile, all launched from one shared interpreter environment. They fall into three families:

Table 3: MCP server registrations. “Pattern” distinguishes native servers from generic CLI-adapter wraps; target paths are written relative to the profile tree.

Server	Pattern	Target	Tools
sagemath	native	skills/research/sagemath-mcp/	10
lean4	native	skills/research/lean4/	4
searxng	native	skills/research/searxng/scripts/	1
vikunja	CLI adapter	skills/productivity/vikunja/	15
wolfram-alpha	CLI adapter	skills/research/wolfram-alpha/	3
lightrag-query	CLI adapter	skills/research/lightrag/	3
siyuan	CLI adapter	skills/productivity/siyuan/	7
zimi	CLI adapter	skills/research/zimi/	4
zotero	CLI adapter	skills/productivity/zotero/	7
scientific-coding	CLI adapter	skills/research/scientific-coding/ (CPU and memory limits via env)	4
latex-manuscript	CLI adapter	skills/productivity/latex-manuscript/	5
hermes-irene	direct script	inside a development worktree since merged and removed: registered, but dangling	—
calibre	native, librarian only	math: environment block without a command; librarian: a server with its own virtual environment	12

The **generic CLI-adapter pattern** is the load-bearing design of this layer. A single script — the generic adapter kept in the profile’s `scripts/` directory — wraps any argparse-based Python CLI tool as a stdio MCP server: it introspects the target’s parser at startup, exposes each subcommand as an MCP tool with JSON-schema arguments, and reconstructs command lines from tool calls. Most of the registered servers use it; adding a new service-backed skill therefore requires only a thin CLI script plus a config entry (endpoint and token via environment), with no protocol code.

Two cross-profile facts follow directly from the table:

- Every path in every MCP registration — including all of the librarian’s — points into the **math profile’s directory tree**: every adapter tool script, every native server script, and the adapter itself. The librarian has no `scripts/` directory of its own. Deleting or moving the math profile would silently break every tool-bridged capability of the librarian.
- Both profiles register `hermes-irene` against a script inside a development worktree that was merged into the main `Irene` package and then removed; the file was never committed, so the registration dangles. It is dormant in math and left enabled in the librarian, where it fails at startup. The correct target is the merged package, where no such script has yet been created.

7 Tool Stack and Service Plane

Table 4 lists the services both profiles reach, grouped by role in the research workflow. Endpoints and credentials are not part of the design: each profile resolves them from its own environment file at load time, so the same registrations serve any deployment. The profiles share the bulk of their environment keys; the math-only keys are messaging and auxiliary settings (Telegram bot token and allowlist, WhatsApp mode fields, an OpenRouter key).

Table 4: Service plane, grouped by role in the research workflow. Both profiles share the same registrations; differences are noted.

Service	Role in the workflow	Integration
LightRAG	Graph-RAG over the curated literature lake: ingestion and multi-hop query	MCP
SearXNG	Federated web metasearch with structured output	MCP
ZIMI	Offline encyclopedia for archival grounding before any live search	MCP
Vikunja	Task ledger: hypothesis trees, phase tracking, cross-session state	MCP and plugin
SiYuan	Block-level notes: human-readable findings and equation blocks	MCP and plugin
Calibre	Ebook library management and ingestion	librarian only
Zotero	Bibliography, BibTeX export, citation verification	MCP
Wolfram Alpha	Symbolic and numeric computation beyond the local stack	MCP
Honcho	Persistent cross-session research memory	built-in memory provider
LM Studio	Local inference for both profiles	model provider
Portainer	Container management of the service plane itself	terminal
Semantic Scholar, arXiv, Crossref	Citation-graph layer and the metadata verification gate	skills

The **four integration channels** a skill may use, in the preference order codified by `math-research-workflow`:

1. *MCP native / adapter tools* — fastest path into services; no shell involved.
2. *Terminal invocations of CLI scripts* bundled with skills (e.g. the arXiv research script, wiki ingestion pipeline).
3. *Built-in Hermes tools* — web search/extraction, file I/O, browser automation.
4. *Local binaries from the sandbox home* (SageMath, Lean 4 via `elan/rustup`, TeXLive) for computation and compilation that must stay on-machine.

Plugins

Plugins are the third extension channel beside skills and MCP servers, and their configuration is byte-identical in the two profiles.

Three facts matter for the abstraction design. First, the librarian enables both plugins while its own `plugins/` directory is empty: plugin resolution falls back to the shared pool exactly as skill resolution does, so the

Table 5: Plugin configuration.

Element	State
Enabled plugins	<code>vikunja</code> , <code>wiki-browser</code> — identical in both profiles
Disabled plugins	<code>hermes-android</code> , <code>spotify</code> — identical in both profiles
Tool-override entries	<code>hermes-android</code> and <code>wiki-browser</code> , both with overrides denied
Profile plugin directory	<code>math</code> carries the plugin trees; the librarian's is empty
Shared installs	resolved from the shared Hermes plugin pool when a profile has no local copy
Desktop surfaces	<code>desktop-plugins/</code> carries the <code>vikunja</code> and <code>wiki-browser</code> surfaces

librarian's plugin capability is entirely borrowed — and, as with skills, the coupling is by name rather than by manifest. Second, `spotify` is listed as disabled in both profiles although no installed copy exists in either profile tree or in the shared pool; the name matches an upstream catalog plugin inside the Hermes source tree, so the entry is a no-op rather than a broken install — the same class of residue as the curator tombstones noted in Section 5. Third, `hermes-android` sits in `math's plugins/` directory but is disabled, while the plugin that carries credentials (`vikunja`, whose dashboard ships a co-located `.env` file) is enabled in both profiles: plugin credentials, like skill credentials, live inside the tree that ships them, not in a profile-level secret store.

8 The Research Workflow

The designated workflow for math projects is the `math-research-workflow` skill (v1.6), a single-agent adaptation of the MathAgent multi-agent system with Graph-of-Agents message passing and Reflexion quality gates. Its six stages, each gated by a 5-check Reflexion gate:

Supporting state: a per-project failure scratchpad under `~/.hermes/profiles/math/scratchpad/`, per-project verified-source registries for citation dedup, and the cron jobs (weekly arXiv scan in the five-topic overlap set; bi-weekly wiki maintenance) that feed Stage 2 continuously. The Irene package (`~/Code/Python/Irene/`) is the shared numerical backend for polynomial optimization experiments across projects (moment/SOS hierarchies, SONC relaxations), invoked from Irene's venv rather than through MCP — the intended `hermes-irene` bridge being the broken one documented in Section 6.

9 Identified Couplings and Defects

Ranked by severity, with the evidence for each entry given in the consequence column:

Table 6: The six stages of the designated research workflow, each gated by a five-check Reflexion gate.

Stage	Activity	Primary tools
1. Problem decomposition	Restate the problem; open Vikunja tasks and a hypothesis tree (conjecture = parent, attack branches = children with status)	vikunja MCP
2. Literature synthesis	Local-first retrieval cascade: LightRAG → academic-research-hub → Semantic Scholar → SearXNG → ZIMI; every citation passes the <i>Citation Verification Gate</i> (arXiv API / Crossref / S2 before any bibliographic fact is recorded)	lightrag, searxng, zimi, s2 skill
3. Computation	Deterministic-first: SymPy → SageMath → Wolfram Alpha → sandboxed scientific-coding; Error-Feedback Loop with a 3-retry budget, exact prior error carried into each retry	sympy-mcp, sagemath-mcp, wolfram-alpha, scientific-coding
4. Formal verification	Three-stage Lean pipeline: formalize → compile clean (<code>lean4_check</code> against an explicit Mathlib project) → statement-fidelity diff vs. the informal claim; only then is a theorem ledger-marked “proven & Lean-verified”	lean4, lean4-goedel-agent
5. Manuscript production	Drafter → Enhancer → Reviewer pass over LaTeX with the ghasemi-latex-style anti-pattern checklist and notation audit	latex-manuscript MCP, ghasemi-latex-style
6. Reflection & close-out	Claims ledger updated; failures promoted to skill pitfalls or project reports; project README + AGENTS.md status synced in the same com-	—

Table 7: Coupling points that bind the profiles to the current project layout.

Coupling	Consequence	Severity
Cross-profile script import (librarian → math)	Every adapter tool script, every native server script and the adapter itself resolve under the math profile; the librarian has no scripts directory of its own	critical
Stale development-worktree path	Both profiles register a tool against a script in a worktree that was merged and removed; dormant in math, failing at startup in the librarian	high
Absolute skill paths in MCP configuration	Every registration hardcodes the full home path; no interpolation mechanism exists	high
Hardcoded service endpoints	Environment files embed host and port directly; no name-based resolution	medium
Project-pinned working directories	Working directory, repository scan roots and execution mode reference specific project trees; the kanban orchestrator is fixed to one profile in both configurations	medium
Toolchain baked into the profile home	The local computation toolchain ships as profile state rather than as a project binding	medium
Stale disabled entries (curator tombstones)	Disabled names that resolve only under the curator archive would silently disable a same-named skill if it were reinstalled	low
Orphaned skill reference	A skill named by the research workflow exists in neither catalog	low
Asymmetric service registration	The same service name means different things in the two profiles	low

10 Toward an Abstraction Layer

The goal is a design in which *profiles are personas and projects are bindings*. A profile declares **what it can do** (capabilities) and *how it behaves* (persona); a project declares **where things live** (paths, endpoints, ledgers). Nothing in today's configuration should need editing to attach the existing math or librarian profile to a new math project.

A. Capability-gated skill activation (replaces negative lists)

Each `SKILL.md` gains one frontmatter field:

```
requires: [computation, formal-proof] # empty list = always available
```

and each profile declares a positive capability set instead of a disabled list:

Persona	Capabilities
mathematician (today: math)	literature, computation, formal-proof, manuscript, knowledge-mgmt, optimization-backends
librarian	literature, ingestion, knowledge-mgmt, manuscripts-light

The loader activates a skill iff `requires ⊆ persona.capabilities`. The librarian's current negative list disappears; tombstone names (entries that resolve only under `.archive/`) become impossible to carry forward, since a capability gate cannot name a skill the persona does not declare; and a new persona is one YAML block, not a diff of every registration.

B. Path interpolation and a shared adapter (removes the critical coupling)

Introduce `${PROFILE_HOME}` and `${HERMES_ROOT}` tokens in `mcp_servers` entries, mirroring the `{baseDir}` mechanism skills already use. Concretely:

1. Move the generic adapter to a shared location: `~/.hermes/tools/mcp_cli_adapter.py` (next to `tirith/uv`, which are already cross-profile). Adapter-wrapped tool scripts move with their skills — they do, since `TOOL_SCRIPT` paths become `${PROFILE_HOME}/skills/...`
2. Each profile's config then references only its own tree; the librarian stops importing from math. Verification is trivial: no path in a profile's config may contain another profile's name.

C. Service registry (removes the endpoint coupling)

A `services.yaml` naming services abstractly, with endpoints resolved from env at load time:

```
1 # ~/.hermes/services.yaml
2 lightrag: { url: ${LIGHTRAG_URL}, key: ${LIGHTRAG_API_KEY} }
3 searxng: { url: ${SEARXNG_URL} }
4 zimi: { url: ${ZIMI_URL} }
5 vikunja: { url: ${VIKUNJA_URL}, token: ${VIKUNJA_TOKEN} }
6 siyuan: { url: ${SIYUAN_URL}, token: ${SIYUAN_TOKEN} }
7 calibre: { url: ${CALIBRE_BASE_URL} }
8 honcho: { url: ${HONCHO_BASE_URL} }
```

MCP entries then reference `$services.lightrag.url` etc. The `.env` files remain the secret store (never committed), but IPs move out of configuration logic into environment, so a service migration is an env edit, not a per-profile config surgery. This also fixes the calibre asymmetry: one definition, activated by capability ingestion.

D. Toolchain bundle (removes the sandbox coupling)

The sandbox home is a *capability artifact* (computation, formal proof and manuscript), not profile state. Split it: keep identity files (`.gitconfig`, `.ssh`, `.bashrc`) in the profile; extract the toolchain into an installable bundle `math-toolchain.tar` or a Docker/ conda-spec manifest that any persona declaring the relevant capabilities can provision on demand — and that a project without formal-proof needs (e.g. pure numerical work) never pays for.

E. Project binding files (the actual project-independence mechanism)

A new math project is then attached by a single file at its root:

```
1 # ~/Code/<NewProject>/.agent.yaml
2 profiles: [math, librarian]
3 working_dir: .
4 vikunja_project: <id> # task ledger target
5 lightRAG_collection: <project> # literature namespace
6 sage_env: sage # toolchain bundle component (conda env name)
```

Profiles discover projects by scanning their `repo_scan_roots`; a profile picks up a project's binding at session start and overrides `terminal.cwd`, ledger targets, and collection names from it. Nothing inside the profiles is edited when the project changes — which is exactly the property today's coupling list (Table 7) shows to be absent.

F. Remediation prerequisites

Before any of A–E is built, four live defects should be fixed in place (each a one-line or one-file change): repoint `hermes-irene` at the merged Irene tree (or disable it for the librarian), prune the tombstone-only names from the disabled lists, remove `simplerag-memory` from `math-research-workflow`'s related-skill list, and give math a real calibre server command or delete its `env-only` block. These are recorded here as open work items; none is applied by this documentation project.

Instantiation model, end state

Figure 2 summarizes the target relationship between templates, personas and projects.

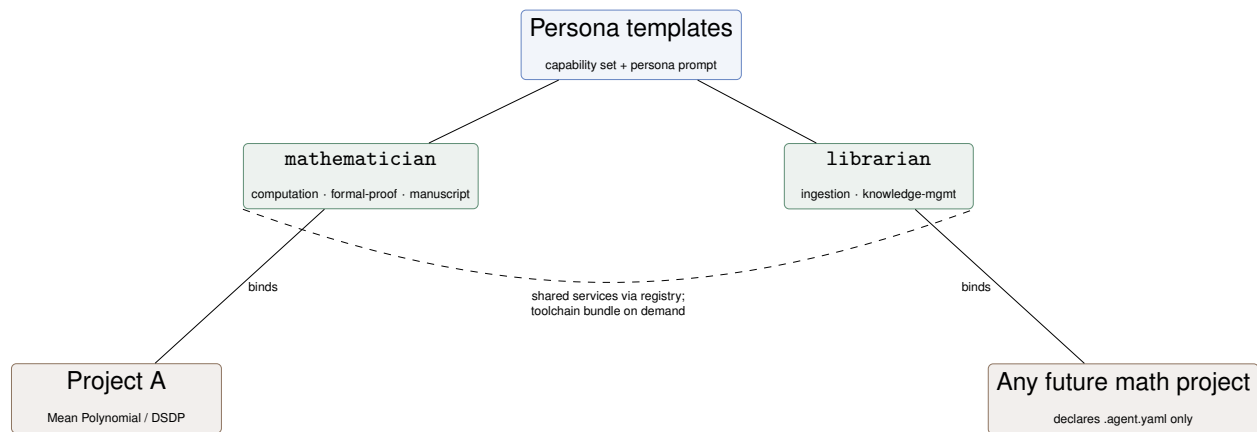


Figure 2: Target model: personas are instantiated from templates with a capability set; projects bind to personas through root-level binding files. No profile-internal configuration changes when the project changes.

1.1 Roadmap for AgentDesign Itself

Phase	Activity	Status
I. Exploration	Read every configuration, skill tree, state database and script; extract the design data programmatically	done
II. Documentation and packs	This document, the sanitized profile packs of Section 4, the skill inventory and the extracted data	this deliverable
III. Remediation PRs	The four one-line fixes of Section 10(F), as separate commits against the live profiles	open
IV. Abstraction pilot	Implement capability frontmatter + loader gate for one skill pair (e.g. <code>sagemath-mcp</code> under a test persona); verify librarian behavior is unchanged	open
V. Migration	Shared adapter move, <code>services.yaml</code> , toolchain bundle; then retire disabled lists	open

A The Self-Hosted Service Plane

Section 7 described the service plane as a design; this appendix records it as a deployment. The `stack/` folder of this project holds one compose project that brings up every container the two profiles talk to. On the live installation those containers run as seven separate compose projects under a single directory, one per subsystem, with credentials written directly into their files; the subject here is the unified version, in which the same sixteen components form one stack whose secrets live in an untracked environment file.

Sixteen components is more than the profiles' service table suggests, because four of them are infrastructure for another component rather than services the profiles call directly: the metasearch limiter store, the graph and vector stores behind the retrieval service, and the database and cache behind the memory service. They are listed here because a stack that omits them is not reproducible. The two inventory tables below therefore divide the components by function, and the tables that follow place them in the workflow.

A.1 Component inventory

Table 8: Component inventory, part one: the edge and the knowledge-management components.

Component	Endpoint	Image	Role in the workflow
npm	:80, :443, :81	jc21/nginx-proxy-manager:latest	Edge of the plane: terminates HTTP/HTTPS on the host, with the routing admin UI on the third port. No proxy hosts are configured yet, so every other component is reached by its own port.
portainer	:9000	portainer/portainer-ce:latest	Management of the plane itself: start, stop, inspect and read the logs of the other containers, driven from the terminal with an API key rather than through a tool bridge.
siyuan	:6806	b3log/siyuan:latest	Block-level note store: findings, synthesis pages and equation blocks, written through the notes tool bridge and the desktop plugin and read back when grounding an answer or drafting a manuscript.
vikunja	:3456	vikunja/vikunja:2.3.0	Task ledger: multi-day work, hypothesis trees (conjecture as parent, attack branches as children carrying a status) and cross-session state.
searxng	:5050	searxng/searxng:latest	Federated metasearch: the live-web layer of the retrieval cascade, returning structured results to the literature stage.
searxng-redis	internal	redis:7-alpine	Limiter store for the metasearch instance: engine bans, suspension timers and request accounting. Not published to the host.

Table 9: Component inventory, part two: retrieval, ingestion and research memory.

Component	Endpoint	Image	Role in the workflow
zimi	:8899	epheterson/zimi	Offline encyclopedia over ZIM archives: sub-second archival grounding for historical and encyclopedic claims, consulted before any rate-limited live search.
lightrag	:9621	ghcr.io/hkuds/ lightrag:latest	Graph-RAG over the curated literature lake: ingests documents and answers multi-hop entity and relation queries. Sole backend of the retrieval tool bridge.
neo4j	:7474, :7687	neo4j:5.20.0	Property-graph store holding the entity graph of the literature lake (browser and Bolt endpoints).
qdrant	:6333, :6334	qdrant/qdrant: latest	Vector store holding the chunk embeddings of the literature lake (HTTP and gRPC endpoints).
paperflow	:8000	built from ./ paperflow	Paper conversion: PDF to Markdown with OCR, feeding the ingestion skills. Jobs are queued and their outputs land in the shared data directory.
pocketbase	:8090	ghcr.io/ muchobien/ pocketbase:latest	Intake queue for the wiki pipeline: a document is dropped here and the pipeline carries it into the knowledge graph and the wiki, decoupling capture from processing.
honcho-api	:8008	built from ./ honcho	Persistent cross-session research memory: the profiles' memory provider, storing what was concluded and serving it back at the start of later sessions.
honcho-deriver	internal	built from ./ honcho	Background worker of the memory service: derives the peer representation and session summaries that the API serves.
honcho-database	127.0.0.1:5433	pgvector/ pgvector:pg15	Store for the memory service: messages, conclusions and their embeddings. Published on the loopback interface only.
honcho-redis	127.0.0.1:6379	redis:8.2	Cache and work queue for the memory service. Published on the loopback interface only.

A.2 Role in the six-stage workflow

Table 10: Where the stack sits in the six-stage workflow of Section 8.

Stage	Components	What they contribute
1. Problem decomposition	vikunja, siyuan	The restated problem becomes a ledger task with a hypothesis tree; notes hold the framing that a ledger entry is too terse to carry.
2. Literature synthesis	zimi, searxng, lightrag, neo4j, qdrant, pocketbase, paperflow	The retrieval cascade runs offline-first: the archive answers what it can, federated search covers the live web, and the graph-RAG service answers multi-hop questions over what has already been ingested. The ingestion components keep that lake current.
3. Computation	none of the stack	Deterministic computation runs against the sandboxed toolchain and the symbolic services; the stack supplies no part of this stage's critical path.
4. Formal verification	none of the stack	The proof assistant runs from the sandbox home against a pinned library project; verification is deliberately local and offline.
5. Manuscript production	siyuan	Manuscript sources are compiled locally; the note store carries the notation decisions and equation blocks that the drafting passes reuse.
6. Reflection and close-out	vikunja, honcho-api	Tasks are closed in the ledger and the outcome is written to the memory service, so the next session starts from what was decided rather than from what was merely said.
Every stage	npm, portainer, honcho-api	Edge access to the plane, container management while working, and the memory that spans sessions.

A.3 Dependencies outside the stack

Table 11: Dependencies the stack does not containerise, and why.

Dependency	Where it lives	Why it is out of scope
Inference server	another machine on the LAN, port 1234	Serves every model call for both profiles and for the memory service. Not containerised because it needs a GPU and multi-gigabyte model weights; both profiles fail loudly when it is unreachable.
Calibre server	systemd service on the host, port 9876	The ebook library and its conversion tooling. The library pre-dates the container stack, and the ingestion skills call it over HTTP.
Zotero	SaaS API	Bibliography, BibTeX export and the citation library.
Wolfram Alpha	SaaS API	Symbolic and numeric computation beyond the local stack, used in the computation stage.
Semantic Scholar, arXiv, Crossref	SaaS APIs	Citation graph and the metadata verification gate that every recorded citation must pass.
OpenRouter	SaaS API	LLM and embedding provider behind the graph-RAG service.

The first two rows are reached over the network exactly as the containers are; the rest are public APIs. Their absence from the stack is a design decision rather than an oversight: each needs hardware, data or a service that a container cannot supply, and each is reachable from the profiles on the same terms as the containerised components.

A.4 Deployment notes

The stack is a single compose project, and no credential is stored in it: every one is interpolated from an untracked environment file whose template ships with the project as a list of placeholders. Bind-mounted service data resolves under one configurable root, so the same file serves both a fresh install and a migration onto existing data, while the memory service's database and cache use named volumes. Two images are built from source and their build contexts are supplied; the retrieval service runs its published upstream image, with the local-build variant given as a commented alternative.

The stack was validated against the live installation rather than merely parsed. The compose configuration resolves cleanly, a dry run produces the complete creation plan, and that plan matches the running containers one for one — identical container names and identical published host ports for all sixteen components.

B The Model Context Protocol Bridge

Section 6 described the bridge between the profiles and their tools; this appendix enumerates what crosses it. Every capability the profiles reach without a shell goes through a Model Context Protocol server, and every server is registered in the profile's configuration under a name that becomes the first half of the tool name the model sees. Thirteen servers are registered in each profile, exposing seventy-five tools in total, and they divide into two families by construction rather than by purpose.

B.1 Conventions of the bridge

Three conventions govern the whole layer, and each of them is visible in the tables below.

- **Naming.** A tool is addressed as `mcp_` followed by the server name and the tool name, each pair separated by a double underscore — for example the task-creation tool of the ledger server. The name is derived from the registration, not chosen by the server, so renaming a server renames every tool it exposes.
- **Families.** A *native* server speaks the protocol directly: it implements the tool list and the call handler itself. A *CLI-adapter* server is a thin wrapper around an ordinary command-line tool — the adapter introspects the tool's argument parser at startup, publishes each subcommand as a tool with a JSON schema for its arguments, and rebuilds a command line from the call it receives. Nine of the thirteen servers are adapters, and the pattern is why adding a new service-backed capability costs a script and a configuration entry rather than protocol code.
- **Credentials.** Every server receives its endpoint and secret through environment variables declared in the registration, not in the script. The scripts are therefore portable between profiles and machines, and the same registration block appears verbatim in both profiles.

The servers divide into those that compute, those that verify and those that retrieve — Tables 12 and 13 — and the two tables together account for the whole registration list of either profile.

Table 12: The computing, verifying and retrieving servers.

Server	Family	Tools	Backing service	What it provides
sagemath	native	10	SageMath, local	Exact symbolic and algebraic computation: solving, factoring, simplification, matrices, number fields, and arbitrary scripts
lean4	native	4	Lean 4 with a pinned Mathlib project	Formal verification: check a file, search the library, run ad-hoc code, attempt an automated proof
wolfram Alpha	CLI adapter	3	Wolfram Alpha SaaS API	Queries whose answer is a number or a closed form rather than a derivation
scientific-coding	CLI adapter	4	Sandboxed Python runtime	Deterministic experiments under CPU and memory caps, with an option to publish results to the note store
searxng	native	1	SearXNG container (Appendix A)	Live federated web search returning structured results
zimi	CLI adapter	4	ZIMI container over ZIM archives	Offline encyclopedia lookups for archival grounding before any live search
lightrag-query	CLI adapter	3	LightRAG container	Multi-hop entity and relation queries over the ingested literature lake

Table 13: The knowledge, ledger and production servers.

Server	Family	Tools	Backing service	What it provides
vikunja	CLI adapter	15	Vikunja container	The task ledger: projects and tasks, with statuses, due dates and comments; the substrate of hypothesis trees
siyuan	CLI adapter	7	SiYuan container	Block-level notes: create, read, search and update documents and blocks, including equations
zotero	CLI adapter	7	Zotero API	Bibliography: search the library, add by DOI, export BibTeX and synchronise a bibliography file
latex-manuscript	CLI adapter	5	Local TeX Live	Manuscript production: compile, AST-aware edit checks, glossary, notation audit and diff-patch
calibre	native	12	calibre-server on the host	Library management and format conversion; registered in the librarian, environment-only in math
hermes-irene	direct script	—	A deleted development worktree	Registered in both profiles but not resolvable: dormant in math, failing at start-up in the librarian

B.2 What each server exposes

Each server is described below in the order the tables introduce it: what it exposes, and how the workflow actually spends it.

sagemath Ten tools over a local SageMath installation: `sage_solve`, `sage_diff`, `sage_integrate`, `sage_simplify`, `sage_factor`, `sage_expand`, `sage_latex`, `sage_matrix`, `sage_number_field` and `sage_run_script`. This is the first stop of the computation stage, and the one that carries exactness: a computation that can be done symbolically is done here rather than numerically, and the `sage_latex` tool exists so that a verified expression can be moved into a manuscript without retyping it.

lean4 Four tools against a pinned library project: a file checker, a library search, an ad-hoc evaluation entry point and an automated prover. The verification stage runs on this server alone, and its discipline is what makes the stage worth having — a check must name its project directory explicitly, because a check without a library context compiles against nothing and proves nothing.

wolfram-alpha Three tools wrapping the public API: a short answer, a validation call and a verification call. It is used for questions whose answer is a closed form or a number, in particular when the local stack is unavailable or when an independent implementation is wanted as a cross-check on a SageMath result.

scientific-coding Four tools running Python inside a sandbox with a CPU-second and memory ceiling declared in the registration. This is where numerical experiments, plots and simulations run when they must not touch the host, and one of the tools writes a result into the note store, which is how an experiment's output reaches the knowledge base without a shell command.

searxng A single search tool over the federated metasearch container. It is the live-web arm of the retrieval cascade and is deliberately last in preference order: the cascade tries local sources first and reaches the open web only when they cannot answer.

zimi Four tools over the offline archive container: search, retrieval, reading a document and suggesting a follow-up. It is the archival-grounding step — encyclopedic and historical facts are settled here, in under a second, before a rate-limited web search is spent on them.

lightrag-query Three tools over the graph-RAG container: a query, a streaming query and a data query. It answers multi-hop questions across the ingested literature lake and is the only way the profiles reach the entity graph, which makes the ingestion pipeline a prerequisite for the literature stage rather than a parallel activity.

vikunja Fifteen tools, the largest surface in the bridge, split between projects (list, search, get, create, update) and tasks (list, overdue, search, get, create, update, delete, complete, log). It carries the first and last stages of the workflow: the decomposition stage opens a task and a hypothesis tree through it, and the close-out stage records the outcome in the same structure. Its second role is continuity — the ledger, not the conversation, is what a later session reads to find out what was decided and what is still open.

siyuan Seven tools over the note store: create, read, update and search documents and blocks. It is the human-readable layer of the knowledge base, used when a finding should be legible rather than machine-queryable, and its equation blocks are how a verified formula survives from the computation stage into a manuscript draft.

zotero Seven tools over the bibliography: searching the library, adding an entry by identifier, listing collections, exporting BibTeX and synchronising a bibliography file. It sits immediately after the citation verification gate in the literature stage — a reference is verified against primary metadata first and only then added to the library, so the library never accumulates a citation that has not been checked.

`latex-manuscript` Five tools over the local TeX installation: compile, an abstract-syntax-aware edit check, glossary generation, a notation audit and a diff-patch applier. This is the manuscript stage's whole tool surface, and its two unusual members are the ones that matter: the edit check catches structural damage that a successful compile would not, and the notation audit catches the same symbol carrying two meanings across sections.

`calibre` Twelve tools for library management and format conversion, registered as a native server in the librarian and as an environment-only block in math — the same name with different meanings in the two profiles, noted in Section 9. Its role is ingestion: converting an acquired book or paper into a form the knowledge base can absorb.

`hermes-irene` Not a working server. Both profiles register it against a script inside a development worktree that was merged into the main package and then deleted, so the registration dangles: dormant in math, enabled and failing at start-up in the librarian. It is listed here because the bridge's inventory is only complete if its one broken member is in it, and because the correct replacement — a bridge to the numerical backend the profiles invoke by shell today — remains the open item recorded in Section 9.

Table 14: Where the bridge enters the six-stage workflow of Section 8.

Stage	Servers	How the stage uses them
1. Problem decomposition	vikunja	The restated problem becomes a task with a hypothesis tree; the tree is what later stages attach results to
2. Literature synthesis	zimi, searxng, lightrag-query, zotero	Archival grounding first, then live search, then multi-hop queries over the ingested lake; a verified reference is added to the bibliography
3. Computation	sagemath, wolfram-alpha, scientific-coding	Exact symbolic work first, an independent implementation as cross-check, sandboxed numerics where the computation must be approximate
4. Formal verification	lean4	Formalise the claim, compile it against a pinned library project, then diff the verified statement against the informal one
5. Manuscript production	latex-manuscript, siyuan	Compile and structurally check the source; the note store carries the notation decisions and equation blocks the draft reuses
6. Reflection and close-out	vikunja, siyuan	Close the tasks, write the outcome into the ledger and the notes, and record the failures that should become skill pitfalls

B.3 Notes on the bridge

The bridge is not capability-gated. Both profiles register the same thirteen servers, and all of them are enabled in both. What differs between the profiles is which *skills* are active, not which tools exist: the librarian's skill list disables the computational and proof surface, yet the librarian's configuration still carries a live handle to it. Capability gating at the skill layer and capability gating at the bridge layer are two different things, and only the first is implemented today.

Seventy-five tools are exposed. The count is dominated by the ledger (fifteen) and the library server (twelve); the two servers that carry the intellectual weight of the workflow, the computer-algebra system and the proof assistant, expose ten and four. Tool count is therefore a poor proxy for importance here — a point the abstraction layer of Section 10 has to respect, since a capability gate keyed on the bridge would grant far more than the workflow needs.

One capability of the workflow is not on the bridge at all. Cross-session memory is served by a built-in provider rather than an MCP server, so it is absent from the inventory above by construction: the memory

service of Appendix A is reached by the runtime, not by a tool call. The same is true of web search and file access, which are built-in tools rather than registered servers.

Every path in the bridge resolves into the math profile. The registrations of both profiles — including the librarian's, which has no script directory of its own — point at scripts inside the math profile's tree. This is the cross-profile coupling of Section 9 seen from the bridge's side: the bridge is the single largest thing that would break if the math profile were moved or deleted.

Sources and Verification

Every statement in this document is derived from a live installation of the two profiles rather than from recollection: the configuration files, environment files, profile metadata, scheduled-job definitions, script trees, plugin trees, sandbox homes and complete skill trees of both profiles were read directly, together with the shared Hermes plugin pool and the shared MCP interpreter environment. The extraction is reproducible — the script that produced the comparison data and the skill inventory ships in this project, and the same script rebuilds the two profile packs of Section 4.

- **Extraction** — a script in `references/` reads both profiles and writes the comparison data and the per-entry skill inventory (`skill_inventory.md`).
- **Packs** — the pack builder in `references/` produces the `math/` and `librarian/` folders from the same sources (`build_profile_packs.py`).
- **Structural claims** — the cross-profile path coupling, the dangling registration and the curator tombstones were each confirmed by direct filesystem and configuration inspection, not inferred.

Redaction note: no credential, endpoint, hostname or personal identifier appears anywhere in this document or in the shipped packs. Secrets are replaced by placeholders in the packs and masked in the extracted data artifacts; the redaction pass is applied mechanically by the pack builder and verified by a residual-secret scan over the result.